

Chapter 6

Synchronization Tools



MD MAHMUDUR RAHMAN

Assistant Professor, IIT

mahmud@juniv.edu

Contents

1. Introduction
2. Critical Section Problem
3. Requirements of Synchronization Mechanism
4. Lock Variable Mechanism
5. Peterson's Solution

Introduction

- When two or more process cooperates with each other, their order of execution must be preserved, otherwise there can be conflicts in their execution and inappropriate outputs can be produced.
- A **cooperative process** is the one which can affect the execution of other process or can be affected by the execution of other process.
- Such processes need to be synchronized so that their order of execution can be guaranteed.
- The **procedure** involved in preserving the appropriate order of execution of *cooperative processes* is known as **Process Synchronization**.
- There are various synchronization mechanisms that are used to synchronize the processes.

Continue...

Race Condition

- A Race Condition typically occurs when two or more threads try to read, write and possibly make the decisions based on the memory that they are accessing **concurrently**.

Critical Section

- The *regions of a program* that try to *access shared resources* and may **cause race conditions** are called critical section.
- To **avoid race condition** among the processes, we need to assure that only **one process at a time** can execute within the critical section.

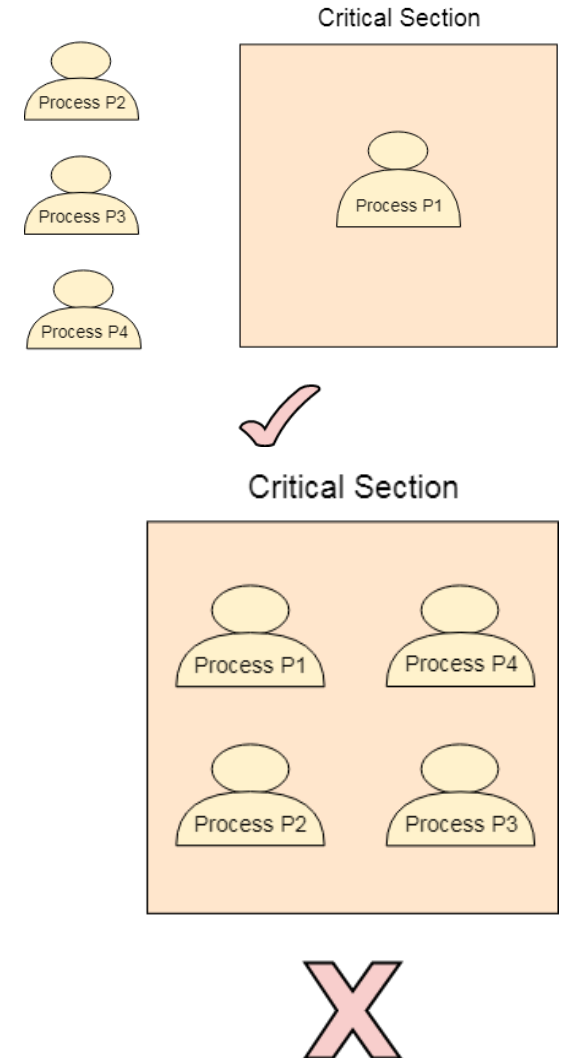
Critical Section Problem

- Critical Section is the part of a program which tries to access shared resources. That resource may be any resource in a computer like a memory location, Data structure, CPU or any IO device.
- The critical section cannot be executed by more than one process at the same time; operating system faces the difficulties in allowing and disallowing the processes from entering the critical section.
- The critical section problem is used to design a set of protocols which can ensure that the Race condition among the processes will never arise.
- In order to synchronize the cooperative processes, our main task is to solve the critical section problem. We need to provide a solution in such a way that the following conditions can be satisfied.

Requirements of Synchronization Mechanisms

Primary

- **Mutual Exclusion:** Our solution must provide mutual exclusion. By Mutual Exclusion, we mean that if one process is executing inside critical section then the other process must not enter in the critical section.
- **Progress:** Progress means that if one process doesn't need to execute into critical section then it should not stop other processes to get into the critical section.



Requirements of Synchronization Mechanisms

Secondary

- **Bounded Waiting:** We should be able to predict the waiting time for every process to get into the critical section. The process must not be endlessly waiting for getting into the critical section.

Lock Variable Mechanism

Lock Variable

- This is the simplest synchronization mechanism. This is a software mechanism implemented in user mode. This is a busy waiting solution which can be used for more than two processes.
- In this mechanism, a Lock variable **lock** is used. Two values of lock can be possible, either 0 or 1. Lock value 0 means that the critical section is vacant while the lock value 1 means that it is occupied.
- A process which wants to get into the critical section first checks the value of the lock variable. If it is 0 then it sets the value of lock as 1 and enters into the critical section, otherwise it waits.
- If we look at the Pseudo Code, we find that there are three sections in the code. Entry Section, Critical Section and the exit section.

```
Entry Section →  
While (lock != 0);  
Lock = 1;  
//Critical Section  
Exit Section →  
Lock = 0;
```


Lock Variable Mechanism

- Initially the value of lock variable is 0. The process which needs to get into the critical section, enters into the entry section and checks the condition provided in the while loop.
- The process will wait infinitely until the value of lock is 1 (that is implied by while loop). Since, at the very first time critical section is vacant hence the process will enter the critical section by setting the lock variable as 1.
- When the process exits from the critical section, then in the exit section, it reassigns the value of lock as 0.

```
Entry Section →  
While (lock != 0);  
Lock = 1;  
//Critical Section  
Exit Section →  
Lock = 0;
```

Lock Variable Mechanism

Every Synchronization mechanism is judged on the basis of four conditions.

- Mutual Exclusion
- Progress
- Bounded Waiting
- Portability

Out of the four parameters, Mutual Exclusion and Progress must be provided by any solution. Lets analyze this mechanism on the basis of the above mentioned conditions.

- **Mutual Exclusion:** The lock variable mechanism doesn't provide Mutual Exclusion in some of the cases. This can be better described by looking at the pseudo code by the Operating System point of view I.E. Assembly code of the program. Let's convert the Code into the assembly language.

```
1. Load Lock, R0          ; entry section
2. CMP R0, #0
3. JNZ 1                   ; if Lock != 0, keep looping
4. Store #1, Lock          ; acquire lock (enter CS)
5. Store #0, Lock          ; release lock (exit CS)
```

Lock Variable Mechanism

- Now let's suppose that processes P1 and P2 are competing for Critical Section and their sequence of execution be as follows (initial value of Lock = 0) -
- P1 executes statement 1 and gets pre-empted.
- P2 executes statement 1, 2, 3, 4 and enters Critical Section and gets pre-empted.
- P1 executes statement 2, 3, 4 and also enters Critical Section.
- Here initially the R0 of process P1 stores lock value as 0 but fails to update the lock value as 1. So when P2 executes it also finds the LOCK value as 0 and enters Critical Section by setting LOCK value as 1. But the real problem arises when P1 executes again it doesn't check the updated value of Lock. It only checks the previous value stored in R0 which was 0 and it enters critical section.
- Now, we got two processes in the critical section. According to the condition of mutual exclusion, more than one process in the critical section must not be present at the same time. Hence, the lock variable mechanism doesn't guarantee the mutual exclusion.
- The problem with the lock variable mechanism is that, at the same time, more than one process can see the vacant tag and more than one process can enter in the critical section. Hence, the lock variable doesn't provide the mutual exclusion that's why it cannot be used in general.
- Since, this method is failed at the basic step; hence, there is no need to talk about the other conditions to be fulfilled.

Peterson's Solution

- This is a software mechanism implemented at user mode.
- It is a busy waiting solution which can be implemented for only two processes.
- It uses two variables that are turn variable and interested variable.
- Our previous solution is affected by mutual exclusion problem.
- However, the Peterson solution provides you all the necessary requirements such as Mutual Exclusion, Progress, Bounded Waiting and Portability.

Analysis of Peterson's Solution

- Let us consider two cooperative processes P0 and P1.
- The entry section and exit section are shown below.
- Initially, the value of interested variables and turn variable is 0.
- Initially process P0 arrives and wants to enter into the critical section.
- It sets its interested variable to True (instruction line 3) and also sets turn to 1 (line number 4).
- Since the condition given in line number 5 is completely satisfied by P0, therefore it will enter in the critical section.

```
voidEntry_Section (int process)
{
    1. int other;
    2. other = 1-process;
    3. interested[process] = TRUE;
    4. turn = process;
    5. while (interested [other] =True && TURN=process);
}
```

Critical Section

```
voidExit_Section (int process)
{
    6. interested [process] = FALSE;
}
```

P0 → 1 2 3 4 5 CS

Analysis of Peterson's Solution

- Meanwhile, process P0 got preempted and process P1 got scheduled.
- P1 also wants to enter in the critical section and executes instructions 1, 2, 3 and 4 of entry section.
- On instruction 5, it got stuck since it doesn't satisfy the condition (value of other interested variable is still true). Therefore it gets into the busy waiting.

P1 → 1 2 3 4 5

- P0 again got scheduled and finish the critical section by executing the instruction no. 6 (setting interested variable to false).
- Now if P1 checks then it are going to satisfy the condition since other process's interested variable becomes false.
- P1 will also get enter the critical section.

```
voidEntry_Section (int process)
{
    1. int other;
    2. other = 1-process;
    3. interested[process] = TRUE;
    4. turn = process;
    5. while (interested [other] =True && TURN=process);
}
```

Critical Section

```
voidExit_Section (int process)
{
    6. interested [process] = FALSE;
}
```

P0 → 6

P1 → 5 CS

Analysis of Peterson's Solution

- Any of the process may enter in the critical section for multiple numbers of times. Hence the procedure occurs in the cyclic order.
- **Mutual Exclusion:** The method provides mutual exclusion for sure. In entry section, the while condition involves the criteria for two variables therefore a process cannot enter in the critical section until the other process is interested and the process is the last one to update turn variable.
- **Progress:** An uninterested process will never stop the other interested process from entering in the critical section. If the other process is also interested then the process will wait.
- **Bounded waiting:** In Peterson solution, a deadlock can never happen because the process which first sets the turn variable will enter in the critical section for sure. Therefore, if a process is preempted after executing line number 4 of the entry section then it will definitely get into the critical section in its next chance.
- **Portability:** This is the complete software solution and therefore it is portable on every hardware.

THANK YOU

